Experiment 05

Ridge and Lasso Regularization

Machine Learning Laboratory

Key Concept

Learning Flow

Underfitting → Good Fit → Overfitting → Regularization → Ridge / Lasso

1. Aim

To understand and implement Ridge and Lasso Regularization for controlling overfitting in regression models and to visualize the effects of underfitting, good fitting, and overfitting.

2. Objectives

After completing this experiment, students should be able to:

1. Understand underfitting, good fit, and overfitting.
2. Understand the effect of increasing polynomial model complexity.
3. Understand why regularization is required.
4. Implement Ridge Regression using L2 regularization.
5. Implement Lasso Regression using L1 regularization.
6. Understand the role of the regularization parameter λ.
7. Understand that scikit-learn uses `alpha` for the regularization parameter.
8. Visualize how regularization controls an over-complex model.
9. Compare regression models using evaluation metrics.

3. Theory

3.1 Model Complexity

A regression model can have different levels of complexity.

A linear regression model can be represented as:

$$\hat{y} = w_0 + w_1 x$$

A polynomial regression model can be represented as:

$$\hat{y} = w_0 + w_1 x + w_2 x^2 + \cdots + w_d x^d$$

where d represents the degree of the polynomial.

As the degree increases, the model becomes more flexible and can represent more complex patterns.

Key Concept

Increasing model complexity is not always beneficial.

$$\text{Too Simple} \rightarrow \text{Underfitting}$$

$$\text{Appropriate Complexity} \rightarrow \text{Good Fit}$$

$$\text{Too Complex} \rightarrow \text{Overfitting}$$

3.2 Underfitting

Underfitting occurs when the model is too simple to capture the important pattern in the data.

For example, if the actual relationship between x and y is curved but a straight line is used, the model may fail to represent the relationship.

Characteristics of underfitting include:

- The model is too simple.
- Important patterns are not learned.
- Training error is relatively high.
- Test error is also generally high.

Example

If the data approximately follows a quadratic relationship but we use:

$$\hat{y} = w_0 + w_1 x$$

the model may underfit the data.

3.3 Good Fit

A model has a good fit when its complexity is appropriate for the underlying pattern in the data.

If the data approximately follows:

$$y \approx x^2$$

a degree-2 polynomial can represent the relationship effectively:

$$\hat{y} = w_0 + w_1 x + w_2 x^2$$

The model captures the main relationship without introducing unnecessary complexity.

3.4 Overfitting

Overfitting occurs when a model becomes too complex and starts learning noise or small fluctuations in the training data.

A high-degree polynomial can be highly flexible and may fit the training observations excessively closely.

Characteristics of overfitting include:

- The model is unnecessarily complex.
- Training error can become very small.
- The model may learn noise.
- Performance on unseen data may be poor.

Important

A model that performs extremely well on training data is not necessarily a good model. The goal of machine learning is to learn patterns that generalize to unseen data.

3.5 Why Regularization?

When a model is too complex, regularization can be used to discourage unnecessarily large coefficients.

The basic idea is:

$$\boxed{\text{New Cost} = \text{Original Cost} + \text{Penalty}}$$

The penalty discourages the model from assigning excessively large values to its coefficients.

Therefore:

$$\text{Regularization} \rightarrow \text{Penalty} \rightarrow \text{Controlled Coefficients} \rightarrow \text{Reduced Model Complexity}$$

3.6 Regularization Parameter

The regularization parameter is commonly represented by λ.

In scikit-learn, this parameter is called `alpha`.

$$\lambda \equiv \texttt{alpha}$$

The parameter controls the strength of the penalty.

Value of λ	**Effect**
Small λ	Weak regularization
Moderate λ	Controlled model complexity
Large λ	Strong regularization
Very large λ	May cause underfitting

Important

A larger value of λ is not always better.
If regularization is too strong, the model can become too simple and underfit the data.

3.7 Ridge Regression

Ridge Regression uses an L2 penalty.

The regularized cost function is:

$$J_{\text{Ridge}} = J + \lambda \sum_{j=1}^{n} w_j^2$$

where:

- J is the original cost.
- λ controls the regularization strength.
- w_j are the model coefficients.

Ridge generally:

- Shrinks large coefficients.
- Controls model complexity.
- Helps reduce overfitting.
- Usually does not make coefficients exactly zero.

Key Concept

Ridge = L2 Regularization

$$\boxed{\text{Penalty} = \lambda \sum w_j^2}$$

Main effect:

$$\boxed{\text{Shrink coefficients}}$$

3.8 Lasso Regression

Lasso Regression uses an L1 penalty.

The regularized cost function is:

$$J_{\text{Lasso}} = J + \lambda \sum_{j=1}^{n} |w_j|$$

Lasso can shrink some coefficients exactly to zero.

Therefore, Lasso can also perform feature selection.

Key Concept

Lasso = L1 Regularization

$$\boxed{\text{Penalty} = \lambda \sum |w_j|}$$

Main effects:

$$\boxed{\text{Coefficient Shrinkage} + \text{Possible Feature Selection}}$$

Important

Lasso does not always remove features.
It can make some coefficients exactly zero depending on the data and the regularization strength.

3.9 Ridge vs Lasso

Property	**Ridge**	**Lasso**
Regularization	L2	L1
Penalty	$\lambda \sum w_j^2$	$\lambda \sum \lvert w_j \rvert$
Coefficient effect	Shrinks	Shrinks
Exact zero coefficients	Generally no	Possible
Feature selection	No	Possible

4. Dataset

A small synthetic dataset representing a curved relationship is used.

x	y
1	1.2
2	3.8
3	8.5
4	15.2
5	23.5
6	36.5
7	48.0
8	64.5
9	80.0
10	101.0
11	120.0
12	145.0

The data approximately follows a quadratic relationship:

$$y \approx x^2$$

with small variations.

5. Experiment Flow

The experiment follows this sequence:

$$\boxed{\text{Degree 1}} \rightarrow \boxed{\text{Underfitting}}$$

$$\boxed{\text{Degree 2}} \rightarrow \boxed{\text{Good Fit}}$$

$$\boxed{\text{High Degree}} \rightarrow \boxed{\text{Overfitting}}$$

$$\boxed{\text{High Degree + Ridge}} \rightarrow \boxed{\text{Controlled Complexity}}$$

$$\boxed{\text{High Degree + Lasso}} \rightarrow \boxed{\text{Coefficient Shrinkage}}$$

6. Software Requirements

- Python 3.x
- Jupyter Notebook or Google Colab
- NumPy
- Pandas
- Matplotlib
- Scikit-learn

7. Procedure

1. Create the dataset containing x and y values.
2. Plot the original data points.
3. Fit a degree-1 polynomial model.
4. Observe the underfitting behaviour.
5. Fit a degree-2 polynomial model.
6. Observe the improved fit.
7. Fit a high-degree polynomial model.
8. Observe the overfitting behaviour.
9. Apply Ridge regularization to the high-degree model.
10. Observe how Ridge controls the fitted curve.
11. Apply Lasso regularization to the high-degree model.
12. Observe the effect of Lasso on the fitted curve.
13. Compare the models using regression evaluation metrics.

8. Python Implementation

Step 1: Import Required Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import PolynomialFeatures
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LinearRegression
from sklearn.linear_model import Ridge
from sklearn.linear_model import Lasso

from sklearn.pipeline import Pipeline

from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
```

Step 2: Create the Dataset

```
data = {
    "x": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12],
    "y": [1.2, 3.8, 8.5, 15.2, 23.5, 36.5,
          48.0, 64.5, 80.0, 101.0, 120.0, 145.0]
}

df = pd.DataFrame(data)

X = df[["x"]]
y = df["y"]

print(df)
```

Step 3: Visualize the Original Data

```
plt.figure(figsize=(8, 5))

plt.scatter(X, y, s=60)

plt.xlabel("X")
plt.ylabel("Y")
plt.title("Original Dataset")

plt.grid(True)
plt.show()
```

The data points show an approximately curved relationship.

Step 4: Demonstrate Underfitting

A degree-1 polynomial is equivalent to a linear regression model.

```
model_underfit = Pipeline([
    ("poly", PolynomialFeatures(degree=1)),
    ("linear", LinearRegression())
])

model_underfit.fit(X, y)

x_plot = np.linspace(1, 12, 300).reshape(-1, 1)

y_pred_underfit = model_underfit.predict(x_plot)

plt.figure(figsize=(8, 5))

plt.scatter(X, y, s=60, label="Data")

plt.plot(
    x_plot,
    y_pred_underfit,
    linewidth=2,
    label="Degree 1"
)

plt.xlabel("X")
plt.ylabel("Y")
plt.title("Underfitting - Degree 1")

plt.legend()
plt.grid(True)
plt.show()
```

Important

The straight-line model cannot capture the curved relationship effectively. Therefore, the degree-1 model demonstrates underfitting.

Step 5: Demonstrate a Good Fit

Now use a degree-2 polynomial.

```
model_good = Pipeline([
    ("poly", PolynomialFeatures(degree=2)),
    ("linear", LinearRegression())
])

model_good.fit(X, y)

y_pred_good = model_good.predict(x_plot)

plt.figure(figsize=(8, 5))

```

```
plt.scatter(X, y, s=60, label="Data")

plt.plot(
    x_plot,
    y_pred_good,
    linewidth=2,
    label="Degree 2"
)

plt.xlabel("X")
plt.ylabel("Y")
plt.title("Good Fit - Degree 2")

plt.legend()
plt.grid(True)
plt.show()
```

The degree-2 polynomial captures the main curved relationship much better.

Step 6: Demonstrate Overfitting

Now increase the model complexity.

```
model_overfit = Pipeline([
    ("poly", PolynomialFeatures(degree=15)),
    ("scale", StandardScaler()),
    ("linear", LinearRegression())
])

model_overfit.fit(X, y)

y_pred_overfit = model_overfit.predict(x_plot)

plt.figure(figsize=(8, 5))

plt.scatter(X, y, s=60, label="Data")

plt.plot(
    x_plot,
    y_pred_overfit,
    linewidth=2,
    label="Degree 15"
)

plt.xlabel("X")
plt.ylabel("Y")
plt.title("High-Degree Polynomial Model")

plt.legend()
plt.grid(True)
plt.show()
```

Important

The high-degree polynomial has much greater flexibility than required for this dataset. Such a highly flexible model can learn small fluctuations in the training data and may generalize poorly to unseen data.
This demonstrates the idea of overfitting.

Step 7: Apply Ridge Regularization

Ridge regularization is applied to the high-degree polynomial model.

```
ridge_model = Pipeline([
    ("poly", PolynomialFeatures(degree=15)),
    ("scale", StandardScaler()),
    ("ridge", Ridge(alpha=10))
])

ridge_model.fit(X, y)

y_pred_ridge = ridge_model.predict(x_plot)

plt.figure(figsize=(8, 5))

plt.scatter(X, y, s=60, label="Data")

plt.plot(
    x_plot,
    y_pred_overfit,
    linewidth=2,
    label="High-Degree Model"
)

plt.plot(
    x_plot,
    y_pred_ridge,
    linewidth=2,
    label="High-Degree + Ridge"
)

plt.xlabel("X")
plt.ylabel("Y")
plt.title("Effect of Ridge Regularization")

plt.legend()
plt.grid(True)
plt.show()
```

Ridge uses an L2 penalty to discourage excessively large coefficient values.

Step 8: Apply Lasso Regularization

Lasso regularization is applied to the high-degree polynomial model.

```
lasso_model = Pipeline([
```

```
    ("poly", PolynomialFeatures(degree=15)),
    ("scale", StandardScaler()),
    ("lasso", Lasso(alpha=0.1, max_iter=100000))
])

lasso_model.fit(X, y)

y_pred_lasso = lasso_model.predict(x_plot)

plt.figure(figsize=(8, 5))

plt.scatter(X, y, s=60, label="Data")

plt.plot(
    x_plot,
    y_pred_overfit,
    linewidth=2,
    label="High-Degree Model"
)

plt.plot(
    x_plot,
    y_pred_lasso,
    linewidth=2,
    label="High-Degree + Lasso"
)

plt.xlabel("X")
plt.ylabel("Y")
plt.title("Effect of Lasso Regularization")

plt.legend()
plt.grid(True)
plt.show()
```

Lasso uses an L1 penalty and can make some coefficients exactly zero.

Step 9: Compare All Models

```
plt.figure(figsize=(10, 6))

plt.scatter(X, y, s=60, label="Data")

plt.plot(
    x_plot,
    y_pred_underfit,
    linewidth=2,
    label="Degree 1 - Underfit"
)

plt.plot(
    x_plot,
    y_pred_good,
```

```
    linewidth=2,
    label="Degree 2 - Good Fit"
)

plt.plot(
    x_plot,
    y_pred_overfit,
    linewidth=2,
    label="Degree 15 - High Complexity"
)

plt.plot(
    x_plot,
    y_pred_ridge,
    linewidth=2,
    label="Degree 15 + Ridge"
)

plt.plot(
    x_plot,
    y_pred_lasso,
    linewidth=2,
    label="Degree 15 + Lasso"
)

plt.xlabel("X")
plt.ylabel("Y")
plt.title("Comparison of Regression Models")

plt.legend()
plt.grid(True)
plt.show()
```

Step 10: Compare Model Performance

```
models = {
    "Degree 1": model_underfit,
    "Degree 2": model_good,
    "Degree 15": model_overfit,
    "Degree 15 + Ridge": ridge_model,
    "Degree 15 + Lasso": lasso_model
}

results = []

for name, model in models.items():

    prediction = model.predict(X)

    mse = mean_squared_error(y, prediction)
    rmse = np.sqrt(mse)
    r2 = r2_score(y, prediction)
```

```
18
19      results.append([
20          name,
21          mse,
22          rmse,
23          r2
24      ])
25
26  results_df = pd.DataFrame(
27      results,
28      columns=["Model", "MSE", "RMSE", "R2"]
29  )
30
31  print(results_df)
```

Important

The metrics above are calculated on the available dataset to study fitting behaviour.
Training performance alone should not be used to claim better generalization.
For proper evaluation of generalization, a separate test dataset should be used.

Step 11: Study the Effect of Different Ridge Values

Students can experiment with different regularization strengths.

```
1   alphas = [0.01, 0.1, 1, 10, 100]
2
3   for alpha in alphas:
4
5       ridge = Pipeline([
6           ("poly", PolynomialFeatures(degree=15)),
7           ("scale", StandardScaler()),
8           ("ridge", Ridge(alpha=alpha))
9       ])
10
11      ridge.fit(X, y)
12
13      prediction = ridge.predict(X)
14
15      mse = mean_squared_error(y, prediction)
16      rmse = np.sqrt(mse)
17      r2 = r2_score(y, prediction)
18
19      print(
20          "Alpha:", alpha,
21          "MSE:", round(mse, 4),
22          "RMSE:", round(rmse, 4),
23          "R2:", round(r2, 4)
24      )
```

Students should observe that changing λ changes the strength of regularization.

$$\lambda \uparrow \quad \Rightarrow \quad \text{Penalty Strength} \uparrow$$

However:

$$\lambda \text{ too large} \quad \Rightarrow \quad \text{Possible Underfitting}$$

9. Observation

Model	Behaviour	Observation
Degree 1	Underfitting	Model is too simple.
Degree 2	Good Fit	Captures the main pattern.
Degree 15	High Complexity	Can produce overfitting behaviour.
Degree 15 + Ridge	Regularized	Coefficients are controlled.
Degree 15 + Lasso	Regularized	Coefficients are shrunk; some may become zero.

10. Connection with Gradient Descent

Regularization can be connected to Gradient Descent through the following sequence:

Add Penalty to Cost

↓

Cost Function Changes

↓

Gradient Changes

↓

Parameter Updates Change

↓

Coefficients Become Controlled

For Ridge:

$$J_{\text{Ridge}} = J + \lambda \sum w_j^2$$

For Lasso:

$$J_{\text{Lasso}} = J + \lambda \sum |w_j|$$

Key Concept

Important Connection
Regularization changes the cost function.
Because the cost function changes, its gradient changes. Therefore, the parameter updates during optimization also change.

11. When to Use Ridge and Lasso?

Situation	Suitable Approach
Keep most features and reduce coefficient magnitude	Ridge
Many features may be irrelevant	Lasso
Feature selection is useful	Lasso
Control coefficient magnitude	Ridge
Some coefficients may become zero	Lasso

12. Viva Questions

1. What is underfitting?
2. What is overfitting?
3. What is model complexity?
4. Why can a high-degree polynomial overfit?
5. Why is regularization required?
6. What is the basic idea of regularization?
7. What is λ?
8. What is the name of λ in scikit-learn?
9. What is Ridge Regression?
10. Which regularization does Ridge use?
11. Write the Ridge penalty term.
12. What happens to coefficients in Ridge Regression?
13. What is Lasso Regression?
14. Which regularization does Lasso use?
15. Write the Lasso penalty term.
16. Can Lasso make coefficients exactly zero?
17. Why is Lasso useful for feature selection?
18. Does Ridge usually make coefficients exactly zero?

19. What happens if λ is very large?
20. Is a larger λ always better?
21. How is regularization related to Gradient Descent?
22. Why should training performance alone not be used to judge generalization?

13. Expected Learning

Students should understand the following relationship:

Model Too Simple → Underfitting

Appropriate Complexity → Good Fit

Excessive Complexity → Overfitting

Add Regularization → Control Coefficients

Ridge / Lasso → Reduced Risk of Overfitting

14. Result

The students successfully implemented polynomial regression models and visualized underfitting, good fitting, and high model complexity. Ridge and Lasso regularization were then applied to control the complexity of the high-degree polynomial model.

15. Conclusion

In this experiment, the effect of model complexity on regression was studied using polynomial regression. A low-degree model demonstrated underfitting, while an excessively high-degree model demonstrated the possibility of overfitting.

Ridge and Lasso regularization were then studied as techniques for controlling model complexity by adding penalty terms to the cost function. Ridge uses an L2 penalty to shrink coefficients, while Lasso uses an L1 penalty that can shrink some coefficients exactly to zero.

Thus, regularization helps control excessive model complexity and can reduce the risk of overfitting.